ASSESSMENT TOOL 1 CO1

TOPIC : Application: Online Banking Fraud Detection

NAME : UDHAYAKUMAR U
REG NO : 192424332
COURSE CODE : CSA0606 - DESIGN AND ANALYSIS OF ALGORITHMS

1.) A fraud detection system analyzes transactions recursively:

$T(n)=T(n/2)+1T(n) = T(n/2) + 1T(n)=T(n/2)+1$

Apply the Master Theorem and determine the time complexity. Identify the appropriate case and justify it. Explain how this efficiency impacts real-time fraud detection performance.

Step 1: Write the Given Recurrence Relation

The given recurrence relation is:

$T(n)=T(n/2)+1T(n) = T(n/2) + 1T(n)=T(n/2)+1$

This means:

- The problem size is reduced by half in every recursive call.
- Each recursive call performs only **constant work (1)**.

Step 2: Compare with the Master Theorem

The standard form of the Master Theorem is:

$T(n)=aT(n/b)+f(n)T(n)=aT(n/b)+f(n)T(n)=aT(n/b)+f(n)$

Compare the given recurrence with the standard form.

Given:

$T(n)=T(n/2)+1T(n)=T(n/2)+1T(n)=T(n/2)+1$

Therefore,

- **$a = 1$**
- **$b = 2$**
- **$f(n) = 1 = \Theta(1)$**

Step 3: Find nlogban^{\log_b a}nlogba

Using the formula:

nlogban^{\log_b a}nlogba

Substitute the values.

nlog21n^{\log_2 1}nlog21

Since,

log21=0\log_2 1 = 0log21=0

Therefore,

n0=1n^0=1n0=1

So,

nlogba=1n^{\log_b a}=1nlogba=1

Step 4: Compare f(n)f(n)f(n) with nlogban^{\log_b a}nlogba

We have

f(n)=1f(n)=1f(n)=1

and

nlogba=1n^{\log_b a}=1nlogba=1

Hence,

f(n)=Θ(nlogba)f(n)=\Theta(n^{\log_b a})f(n)=Θ(nlogba)

Both are equal.

Step 5: Identify the Master Theorem Case

Master Theorem has three cases.

Case 1

If

f(n)=O(nlogba−ε)f(n)=O(n^{\log_ba-\varepsilon})f(n)=O(nlogba−ε)

Not satisfied.

Case 2

If

f(n)=Θ(nlogbalogkn)f(n)=\Theta(n^{\log_ba}\log^k n)f(n)=Θ(nlogbalogkn)

Here,

f(n)=Θ(1)f(n)=\Theta(1)f(n)=Θ(1)

and

nlogba=1n^{\log_ba}=1nlogba=1

Since

1=Θ(1)1=\Theta(1)1=Θ(1)

this satisfies **Case 2** with **k = 0**.

Step 6: Apply the Formula

For **Master Theorem Case 2**

$$T(n)=\Theta(n^{\log_b a}\log^{k+1} n)$$

Here,

- $n^{\log_b a}=1$
- $k=0$

Therefore,

$$T(n)=\Theta(1\times\log n)$$

Hence,

$$\boxed{T(n)=\Theta(\log n)}$$

Step 7: Time Complexity

$$\boxed{\Theta(\log n)}$$

The running time grows logarithmically with the input size.

Step 8: Justification

- ❖ At each recursive call, the input size is divided by **2**.
- ❖ Only **constant work (1)** is performed at each level.
- ❖ The recursion continues until the problem size becomes **1**.
- ❖ The total number of recursive levels is:

log2n\log_2 nlog2n

Since each level performs constant work,

1+1+1+⋯(log2n times)1+1+1+\cdots (\log_2 n \text{ times})1+1+1+⋯(log2n times)

Therefore,

T(n)=Θ(logn)T(n)=\Theta(\log n)T(n)=Θ(logn)

Step 9: Recursion Tree (Explanation)

Suppose **n = 16**

```
T(16)
  |
  v
T(8)
  |
  v
T(4)
  |
  v
T(2)
  |
  v
T(1)
```

Levels:

- Level 1 → T(16)
- Level 2 → T(8)
- Level 3 → T(4)
- Level 4 → T(2)
- Level 5 → T(1)

Total levels =

$$\log_2 16 = 4$$

Therefore,

$$T(n)=\Theta(\log n)$$

Step 10: Impact on Real-Time Fraud Detection

A logarithmic time complexity is highly efficient for online banking systems.

- The input size is reduced by half at every recursive step.
- Even if the number of transactions increases significantly, processing time grows very slowly.
- Fraudulent transactions can be detected quickly with low latency.
- The system can handle millions of banking transactions efficiently.
- This improves real-time fraud detection, scalability, and overall system performance.